

Documentation complète - Backend EduSphere

Document d'apprentissage. Objectif : comprendre **tout** le backend, module par module - rôle et fonctionnement de chaque contrôleur, route, service, provider, middleware, modèle, fonction et variable.

Projet : `backend-benjo/EduSphere` · Laravel 12 · PHP 8.2 · PostgreSQL · Auth **JWT**.

Table des matières

- **Partie 0 - Vue d'ensemble**
 - 0.1 Pile technique
 - 0.2 Arborescence des dossiers (rôle de chacun)
 - 0.3 Cycle de vie d'une requête HTTP
 - 0.4 Le multi-tenant (isolation par établissement)
 - 0.5 Conventions de code
- **Partie 1 - Module #0 « Socle » (réalisé par Benjo)**
 - 1.1 Authentification (JWT)
 - 1.2 Le scope d'établissement (middleware)
 - 1.3 RBAC : rôles & permissions
 - 1.4 Workflow « Demandes » (création de comptes)
 - 1.5 Établissements
 - 1.6 Les modèles du socle
 - 1.7 Les tables (migrations) du socle
 - 1.8 Les routes du socle
- **Partie 2 - Module #1 « Structure Académique » (notre lot)**
- **Annexes** - glossaire, format des réponses, codes d'erreur

Partie 0 - Vue d'ensemble

0.1 Pile technique

Brique	Choix	Rôle
Framework	Laravel 12 (PHP 8.2)	structure MVC + API
Base de données	PostgreSQL	stockage relationnel
Authentification	JWT (tymon/jwt-auth)	jetons d'accès sans état (stateless)
Temps réel	Reverb / Pusher	websockets (hérité du template nexachat)
Images	Intervention Image	redimensionnement/traitement
Stockage fichiers	Flysystem S3	documents/médias (local ou S3)
Tests	PHPUnit	tests automatisés

Le projet a été initialisé depuis un template de chat temps réel (nexachat/backend), d'où la présence de Reverb/Pusher. Le code métier EduSphere est construit par-dessus.

0.2 Arborescence des dossiers (rôle de chacun)

```
EduSphere/
app/
  Events/          -> "événements" métier (ex. ConnexionReussie) - signaux diffusés
  Http/
    Controllers/   -> reçoivent la requête HTTP, renvoient une réponse JSON
    Middleware/    -> filtres exécutés AVANT le contrôleur (auth, tenant, permission)
    Requests/      -> "FormRequest" : valident les données entrantes
  Jobs/           -> tâches asynchrones mises en file (ex. journaliser une connexion)
  Models/         -> "modèles" Eloquent : 1 classe = 1 table, + relations & logique
  Concerns/       -> traits réutilisables par les modèles (ex. AppartientEtablissement)
  Scopes/         -> filtres globaux Eloquent (ex. PorteeEtablissement)
  Providers/      -> "service providers" : amorçage de l'application
  Services/       -> la LOGIQUE MÉTIER (le cur) - appelée par les contrôleurs
  Support/        -> utilitaires transverses (ex. ContexteTenant)
bootstrap/
  app.php         -> configuration centrale (routing, middleware, exceptions)
  providers.php   -> liste des service providers à charger
config/          -> fichiers de configuration (auth, database, jwt...)
database/
  factories/      -> générateurs de fausses données (tests/seed)
  migrations/     -> définition des tables (versionnée)
  seeders/        -> remplissage de données de démonstration
routes/
  api.php         -> routes de l'API (préfixe /api)
  web.php         -> routes web (peu utilisées ici)
  console.php     -> commandes artisan personnalisées
```

Le principe d'architecture clé : « contrôleur mince, service épais ». Le contrôleur ne fait que recevoir/valider/répondre ; toute la logique vit dans un **Service**.

0.3 Cycle de vie d'une requête HTTP

Exemple : GET /api/roles (lister les rôles).

1. Le navigateur envoie la requête avec l'en-tête Authorization: Bearer <jwt>
2. bootstrap/app.php applique les middlewares du groupe « api »
3. routes/api.php fait correspondre l'URL à RoleController@index
4. Les middlewares de la route s'exécutent dans l'ordre :
 - a. auth:api -> vérifie le jeton JWT, charge l'utilisateur
 - b. etablisement.scope -> lit utilisateur->etablisement_id et le met dans la requête
 - c. permission:... -> vérifie que l'utilisateur a la permission requise
5. Le contrôleur RoleController@index est appelé
6. Il délègue au PermissionService (logique métier)
7. Le service interroge les Modèles (Role, Permission) -> SQL -> PostgreSQL
8. Le contrôleur renvoie response()->json(['succes' => true, 'donnees' => ...])

Le « fil rouge » à retenir : **route -> middlewares -> contrôleur -> service -> modèle -> BD -> JSON.**

0.4 Le multi-tenant (isolation par établissement)

EduSphere est **multi-tenant** : une seule base sert tous les établissements, chaque établissement (« tenant ») ne voit QUE ses données. La clé est la colonne `etablisement_id`.

Deux mécanismes coexistent :

1. **Côté socle (Benjo)** - le middleware `EtablissementScope` lit `etablissement_id` depuis l'utilisateur authentifié (extrait du **jeton JWT**) et le dépose dans la requête. Chaque contrôleur filtre ensuite **manuellement** (`->where('etablissement_id', ...)`).
2. **Côté nos modules** - un **scope global Eloquent** (`PorteeEtablissement`) filtre **automatiquement** toutes les requêtes des modèles « tenants ». Plus sûr : impossible d'oublier le filtre. (Détailé en Partie 2.)

Important : le tenant n'est **jamais** transmis par en-tête HTTP côté client - il est contenu dans le jeton JWT, donc infalsifiable sans la clé secrète du serveur.

0.5 Conventions de code

- **Nommage en français** : `genererToken()`, `estValide()`, `peut()`, `soumettreEnseignant()`.
- **Réponses JSON uniformes** : toujours `{ "succes": bool, "message": string?, "donnees": any?, "code": string? }`.
- **Logique métier dans les Services**, jamais dans les contrôleurs.
- **Validation dans les FormRequest** (`app/Http/Requests`), pas dans les contrôleurs (sauf cas simples).
- **Modèles** : propriété `$casts` (conversion de types), `boot()` + `Str::uuid()` pour générer l'UUID public, `$fillable` (colonnes assignables en masse), `$hidden` (colonnes masquées dans le JSON).
- **Horodatage** : colonnes Laravel standard `created_at` / `updated_at` (et non `cree_le`/`mis_a_jour_le`).

Partie 1 - Module #0 « Socle » (réalisé par Benjo)

Le socle fournit : l'authentification, l'identité (utilisateurs), les établissements, les rôles/permissions (RBAC), et le workflow de création de comptes. **Tous les autres modules s'appuient dessus.**

1.1 Authentification (JWT)

Qu'est-ce que JWT ?

Un **JSON Web Token** est une chaîne signée que le serveur remet au client à la connexion. Le client le renvoie à chaque requête (`Authorization: Bearer <token>`). Le serveur le vérifie grâce à une **clé secrète** (`JWT_SECRET`) - pas besoin de stocker la session côté serveur (« stateless »). Le jeton contient des « claims » (revendications), ici notamment `etablissement_id` et `role`.

config/auth.php (extrait pertinent)

```
'guard' => env('AUTH_GUARD', 'api'), // le garde par défaut est « api »
'guards' => [
    'api' => [ 'driver' => 'jwt', 'provider' => 'users' ],
],
```

-> Quand on écrit `auth:api` sur une route, Laravel utilise le **driver jwt** pour identifier l'utilisateur à partir du jeton.

app/Services/JwtService.php

Service utilitaire qui **enrobe** la librairie `tymon/jwt-auth`. Rôle : centraliser toutes les opérations sur les jetons.

Méthode	Paramètres	Retour	Rôle / fonctionnement
---------	------------	--------	-----------------------

<code>genererToken</code>	Utilisateur <code>\$utilisateur</code>	string	Crée un jeton d'accès pour cet utilisateur (<code>JWTAuth::fromUser</code>).
<code>genererRefreshToken</code>	Utilisateur <code>\$utilisateur</code>	string	Crée un jeton de rafraîchissement (claim <code>type=refresh</code>) pour obtenir un nouvel accès sans se reconnecter.
<code>utilisateurDepuisToken</code>	-	?Utilisateur	Décode le jeton présent dans la requête et renvoie l'utilisateur ; renvoie <code>null</code> si jeton expiré/invalidé/absent (capture les exceptions JWT).
<code>invaliderToken</code>	-	bool	Invalide (blackliste) le jeton courant - utilisé à la déconnexion. <code>true</code> si réussi.
<code>rafraichirToken</code>	-	?string	Génère un nouveau jeton à partir de l'ancien ; <code>null</code> si échec.

`app/Services/AuthService.php`

Le cur de l'authentification. Injecté avec `JwtService` et `PermissionService` (injection par constructeur - Laravel les fournit automatiquement).

Constantes : - `MAX_TENTATIVES = 5` -> nombre d'essais de connexion avant blocage. - `DUREE_BLOCAGE = 900` -> durée du blocage en secondes (15 min).

login(string \$email, string \$motDePasse, Request \$request): array - déroulé : 1. Construit une clé de limitation `login:<ip>:<email>`. 2. Si trop de tentatives (`RateLimiter::tooManyAttempts`) -> exception `RuntimeException(429)`. 3. Cherche l'utilisateur par email. Si introuvable **ou** mauvais mot de passe (`Hash::check` contre `mot_de_passe_hash`) -> enregistre une tentative, journalise `connexion_echouee` (via `LogConnexionJob`), émet l'événement `ConnexionEchouee`, et lève `RuntimeException(401)`. 4. Si le compte est désactivé (`est_actif=false`) -> `RuntimeException(403)`. 5. Si l'utilisateur a un établissement mais que celui-ci est invalide (`abonnement_inactif`, `estValide()`) -> `RuntimeException(403)`. 6. Réinitialise le compteur de tentatives. 7. Génère **access token + refresh token** (via `JwtService`). 8. Crée une ligne `SessionUtilisateur` (`token`, `ip`, `agent`, `appareil Mobile/Desktop`, `expire_le`). 9. Met à jour `derniere_connexion` (sans toucher `updated_at` -> `updateQuietly`). 10. Journalise `connexion_reussie` (job) et émet `ConnexionReussie`. 11. Charge les **permissions effectives** (via `PermissionService`). 12. Renvoie un tableau : `access_token`, `refresh_token`, `token_type=Bearer`, `expire_dans` (secondes), utilisateur (profil + rôle) et permissions.

logout(Utilisateur, string \$token, Request): void - invalide le jeton JWT, passe la `SessionUtilisateur` correspondante à `est_active=false` + `ferme_le=now()`, journalise `deconnexion`.

rafraichirToken(Utilisateur, string \$ancienToken): array - obtient un nouveau jeton (`JwtService::rafraichirToken`), met à jour la session (nouveau token + nouvelle expiration), renvoie `access_token/token_type/expire_dans`. Lève `RuntimeException(401)` si échec.

moi(Utilisateur): array - charge les relations `role` et `etablissement`, renvoie le profil complet de l'utilisateur connecté + ses permissions effectives. (« qui suis-je ? »)

`app/Http/Middleware/AuthJWT.php`

Middleware d'authentification alternatif (le projet utilise surtout `auth:api`, mais ce middleware fait un travail équivalent + contrôles métier). `handle()` : 1. Récupère l'utilisateur depuis le jeton

(JwtService::utilisateurDepuisToken). 2. null -> réponse 401 UNAUTHENTICATED. 3. Compte désactivé -> 403 ACCOUNT_DISABLED. 4. Établissement à l'abonnement inactif -> 403 SUBSCRIPTION_INACTIVE. 5. Sinon auth()->setUser(\$utilisateur) puis passe au middleware suivant.

app/Http/Controllers/Auth/LoginController.php

Le contrôleur (mince) qui expose l'auth. Injecte AuthService.

Méthode	Route	Rôle
login(LoginRequest)	POST /api/auth/login	Délègue à AuthService::login. En cas de RuntimeException, convertit le code en statut HTTP (avec garde-fou : si le code n'est pas numérique - ex. code SQL - renvoie 500).
logout(Request)	POST /api/auth/logout	Déconnecte l'utilisateur courant (Auth::user()), jeton via bearerToken()).
refresh(Request)	POST /api/auth/refresh	Rafraîchit le jeton ; 401 si invalide.
moi()	GET /api/auth/me	Renvoie le profil + permissions de l'utilisateur connecté.

app/Http/Requests/Auth/LoginRequest.php

FormRequest de validation de la connexion. - authorize(): true -> tout le monde peut tenter de se connecter. - rules(): email (requis, format email), mot_de_passe (requis, min 6). - messages(): messages d'erreur personnalisés en français.

Événements & Job liés à l'auth

- **App\Events\ConnexionReussie / ConnexionEchouee** : objets « signal » émis lors d'une connexion (réussie/échouée). Ils permettent de brancher des réactions (notifications, alertes sécurité) sans alourdir AuthService.
- **App\Jobs\LogConnexionJob** (implements ShouldQueue) : journalise une connexion **de façon asynchrone** (file auth). Propriétés : \$tries=3 (3 essais), \$backoff=5 (5 s entre essais). Le constructeur reçoit utilisateurId, sessionId, typeEvenement, ipAdresse, agentNavigateur, detail. handle() crée une ligne JournalConnexion. Avantage : ne pas ralentir la réponse de connexion par une écriture en base.

1.2 Le scope d'établissement (middleware)

app/Http/Middleware/EtablissementScope.php

C'est le **gardien du multi-tenant** côté socle. Appliqué après auth:api. handle() : 1. Récupère l'utilisateur authentifié : Auth::guard('api')->user(). 2. Si absent -> 401 « Non authentifié ». 3. Si l'utilisateur n'a **pas** d'établissement (etablissement_id === null, ex. super admin) -> 403 « Aucun établissement associé ». 4. Sinon, dépose l'identifiant dans la requête : \$request->attributes->set('etablissement_id', ...). Les contrôleurs le relisent via \$request->attributes->get('etablissement_id').

À retenir : ce middleware **ne filtre pas** les requêtes SQL lui-même ; il se contente de rendre disponible l'etablissement_id. C'est chaque contrôleur du socle qui filtre. (Nos modules, eux, automatisent ce filtrage - voir Partie 2.)

1.3 RBAC : rôles & permissions

RBAC = *Role-Based Access Control*. Un utilisateur a un **rôle** (ex. `admin_universitaire`), un rôle possède des **permissions** (ex. `enseignants.creer`). En plus, on peut accorder ou retirer des permissions à un utilisateur précis (overrides).

Tables impliquées

- `roles` (id, etablissement_id, nom, code, description, est_systeme)
- `permissions` (id, code, libelle, module, description)
- `role_permissions` (pivot rôle permission)
- `utilisateur_permissions` (override : utilisateur permission + type_acces dans {accorder, retirer})

Modèles

- **Role** : \$fillable = etablissement_id, nom, code, description, est_systeme ; cast est_systeme=bool. Relations : etablissement(), permissions() (BelongsToMany via `role_permissions`), utilisateurs() (HasMany). Helper `codesPermissions()` : array -> liste des codes de permission du rôle. Un rôle « système » (`est_systeme=true`) est protégé (non modifiable/supprimable).
- **Permission** : public \$timestamps=false ; \$fillable = code, libelle, module, description. Relations `roles()` et `utilisateurs()` (BelongsToMany).

`app/Services/PermissionService.php`

Le moteur d'autorisation. Utilise un **cache** pour ne pas recalculer les permissions à chaque requête.

Constantes : `CACHE_TTL=300` (5 min), `CACHE_PREFIX='edusphere:permissions:'`.

Méthode	Rôle / fonctionnement
<code>permissionsEffectives(Utilisateur): array</code>	Renvoie la liste des codes de permission de l'utilisateur, mise en cache 5 min (clé <code>prefix+id</code>). Délègue le calcul à <code>calculerPermissions</code> .
<code>calculerPermissions(Utilisateur): array (privé)</code>	(1) permissions du rôle (jointure <code>role_permissions</code>) ; (2) overrides de l'utilisateur (<code>utilisateur_permissions</code>) séparés en <code>accorder/retirer</code> ; (3) résultat = (perms du rôle U accordées) - retirées.
<code>peut(Utilisateur, string \$code): bool</code>	true si le code est dans les permissions effectives. C'est la fonction appelée par le middleware de permission.
<code>peutTout(Utilisateur, array \$codes): bool</code>	true seulement si toutes les permissions de la liste sont présentes.
<code>definirPermission(Utilisateur, \$codePermission, \$typeAcces, \$accordePar): void</code>	Crée/maj un override (<code>updateOrCreate</code> dans <code>utilisateur_permissions</code>) puis invalide le cache de l'utilisateur.
<code>reinitialiserPermission(Utilisateur, \$codePermission): void</code>	Supprime l'override puis invalide le cache.
<code>invaliderCache(int \$id): void</code>	Oublie le cache des permissions d'un utilisateur.
<code>invaliderCacheRole(int \$roleId): void</code>	Oublie le cache de tous les utilisateurs ayant ce rôle.
<code>creerRole(array): Role</code>	Crée un rôle dans une transaction ; synchronise ses permissions si fournies.
<code>mettreAJourRole(Role, array): Role</code>	Refuse si rôle système. Met à jour + resynchronise les permissions + invalide le cache du rôle.

<code>supprimerRole(Role): void</code>	Refuse si rôle système, ou s'il est encore assigné à des utilisateurs.
<code>syncPermissionsRole(Role, array \$codes): void (privé)</code>	Convertit les codes en ids et fait <code>permissions()->sync(...)</code> (avec <code>cree_le</code>).
<code>toutesLesPermissions(): Collection</code>	Toutes les permissions (cache 1 h), triées par module puis code.
<code>permissionsGroupeesParModule(): Collection</code>	Les permissions regroupées par module (pour l'UI).
<code>permissionsUtilisateur(Utilisateur): Collection</code>	Les overrides d'un utilisateur (code, libellé, module, <code>type_acces</code> , date).

app/Http/Middleware/CheckPermission.php

Middleware permission: `(alias). handle(Request, Closure, string ...$permissions) : - Récupère l'utilisateur (auth()->user()), 401 si absent. - Logique OU : si l'utilisateur a au moins une des permissions listées -> passe. - Sinon -> 403 FORBIDDEN (avec la/les permission(s) requises dans la réponse).`

(!) Bug connu dans bootstrap/app.php : l'alias 'permission' pointe (par erreur) sur `EtablissementScope` au lieu de `CheckPermission`. À corriger côté socle.

app/Http/Controllers/Permission/RoleController.php

Expose la gestion des rôles/permissions. Injecte `PermissionService`.

Méthode	Route	Rôle
<code>index</code>	GET /api/roles	Rôles système (<code>etablissement_id</code> null) + rôles de l'établissement courant, avec leurs permissions.
<code>store(CreateRoleRequest)</code>	POST /api/roles	Crée un rôle (rattaché à l'établissement courant).
<code>show(int \$id)</code>	GET /api/roles/{id}	Détail d'un rôle + permissions.
<code>update(CreateRoleRequest, \$id)</code>	PUT /api/roles/{id}	Met à jour (422 si rôle système).
<code>destroy(int \$id)</code>	DELETE /api/roles/{id}	Supprime (422 si système ou assigné).
<code>permissionsUtilisateur(\$id)</code>	GET /api/utilisateurs/{id}/permissions	Overrides + permissions effectives d'un utilisateur.
<code>setPermissionUtilisateur(Request \$id)</code>	POST /api/utilisateurs/{id}/permissions	Accorde/retire une permission à un utilisateur (code + <code>type_acces</code>).
<code>resetPermissionUtilisateur(\$id, \$code)</code>	DELETE /api/utilisateurs/{id}/permissions/{code}	Supprime un override.
<code>toutesLesPermissions()</code>	GET /api/permissions	Catalogue des permissions groupées par module.

1.4 Workflow « Demandes » (création de comptes)

Principe central d'EduSphere : on ne crée pas un compte directement. On **soumet une demande**, puis une autorité la **valide** (et c'est la validation qui crée le compte, avec un mot de passe défini à ce moment-là).

- Une **université** demande sa création -> validée par le **super admin** (crée l'établissement
- le compte `admin_universitaire`).

- Un **admin universitaire** demande un compte **enseignant/étudiant** -> validé par lui-même/super admin (crée le utilisateur avec le bon rôle).

app/Models/DemandeCompte.php

- \$table='demandes_compte', public \$timestamps=false.
- \$fillable : type_demande, donnees, etablissement_id, statut, traite_par, traite_le, motif_rejet, utilisateur_cree_id, etablissement_cree_id.
- \$casts : **donnees** => 'array' (colonne JSON tableau PHP), soumis_le/traite_le en datetime.
- boot() : génère l'uuid à la création.
- Relations : etablissement(), traitePar(), utilisateurCree(), etablissementCree().
- Helpers d'état : estEnAttente(), estValidee(), estRejetee() (comparent statut).
- La colonne **donnees (JSON)** stocke le contenu variable de la demande (infos établissement, infos admin, ou infos enseignant/étudiant) - souple, sans colonnes dédiées.

app/Services/DemandeService.php

Injecte PermissionService.

Méthode	Rôle / fonctionnement
soumettreEtablissement(array): DemandeCompte	Crée une demande type=etablissement, statut=en_attente, sans établissement. Aucun compte créé ici.
soumettreEnseignant(array, int \$etabId): DemandeCompte	Demande type=enseignant rattachée à l'établissement.
soumettreEtudiant(array, int \$etabId): DemandeCompte	Demande type=etudiant.
lister(Utilisateur, array \$filtres, int \$perPage): LengthAwarePaginator	Super admin -> demandes etablissement ; admin -> demandes enseignant/etudiant de son établissement. Filtres statut/type_demande. Trié par soumis_le desc, paginé.
valider(DemandeCompte, string \$mdp, Utilisateur): array	Refuse si déjà traitée (422). Dans une transaction , route selon type_demande (match) vers la bonne méthode privée.
validerEtablissement(...) (privé)	(1) crée l'Etablissement ; (2) récupère le rôle système admin_universitaire ; (3) crée le compte admin (mdp hashé) ; (4) marque la demande validee + liens créés. Retour : {etablissement, admin}.
validerEnseignant(...) (privé)	Récupère le rôle système enseignant, crée le utilisateur rattaché à l'établissement de la demande, marque la demande validée.
validerEtudiant(...) (privé)	Idem avec le rôle etudiant.
rejeter(DemandeCompte, string \$motif, Utilisateur): DemandeCompte	Refuse si déjà traitée ; passe statut=rejetee + motif_rejet.
statistiques(Utilisateur): array	Compteurs en_attente/validees/rejetees/total, filtrés selon le rôle.

Frontière importante pour nos modules #2/#5 : valider* crée le **compte utilisateur** (table utilisateurs) avec le bon rôle. Le **profil métier** (table enseignants / etudiants avec matricule, grade...) reste à créer par **nos** modules.

`app/Http/Controllers/Demande/DemandeController.php`

Injecte `DemandeService`.

Méthode	Route	Accès	Rôle
<code>soumettreEtablissement(DemandeEtablissementRequest)</code>	<code>POST /api/demandes/etablissement</code>	<code>public</code>	Formulaire public du site vitrine.
<code>soumettreEnseignant(DemandeEnseignantRequest)</code>	<code>POST /api/demandes/enseignant</code>	<code>permission:enseignants.create</code>	Admin universitaire.
<code>soumettreEtudiant(DemandeEtudiantRequest)</code>	<code>POST /api/demandes/etudiant</code>	<code>permission:etudiants.create</code>	Admin/secrétaire.
<code>index(Request)</code>	<code>GET /api/demandes</code>	<code>permission:utilisateurs.liste</code>	Clistés selon le rôle.
<code>show(int \$id)</code>	<code>GET /api/demandes/{id}</code>	<code>idem</code>	Détail + contrôle d'accès.
<code>statistiques()</code>	<code>GET /api/demandes/statistiques</code>	<code>idem</code>	Compteurs.
<code>valider(ValiderDemandeRequest, \$id)</code>	<code>POST /api/demandes/{id}/valider</code>	<code>idem</code>	Valide + crée le(s) compte(s).
<code>rejeter(RejeterDemandeRequest, \$id)</code>	<code>POST /api/demandes/{id}/rejeter</code>	<code>idem</code>	Rejette avec motif.
<code>autoriserAcces(DemandeCompte (privé))</code>		-	Vérifie les droits : super admin -> seulement demandes établissement ; admin -> seulement les demandes de son établissement (sinon abort (403)).

1.5 Établissements

`app/Models/Etablissement.php`

- `$fillable` : nom, adresse, ville, pays, telephone, email, est_actif ; cast `est_actif=bool`.
- `boot()` génère l'uuid. Relations : `utilisateurs()`, `roles()`, `demandes()`.
- `estValide()` : bool -> renvoie `est_actif` (utilisé par l'auth pour bloquer un établissement « abonnement inactif »).

`app/Services/EtablissementService.php`

Injecte `PermissionService` (pour invalider les caches lors d'un changement d'état).

Méthode	Rôle
<code>lister(int \$perPage, array \$filtres): LengthAwarePaginator</code>	Liste paginée (super admin). Recherche <code>ilike</code> sur nom/email/ville (insensible à la casse, spécifique PostgreSQL), filtre <code>est_actif</code> , compte les utilisateurs (<code>withCount</code>).
<code>mettreAJour(Etablissement, array): Etablissement</code>	Met à jour et renvoie l'instance rafraîchie.

<code>toggleActif(Etablissement): Etablissement</code>	Bascule <code>est_actif</code> . Effet de bord clé : invalide le cache des permissions de tous les utilisateurs de l'établissement (un établissement désactivé doit bloquer ses comptes).
<code>statistiques(): array</code>	KPIs plateforme : total établissements, actifs, total utilisateurs.

app/Http/Controllers/Etablissement/EtablissementController.php

Injecte `EtablissementService`.

Méthode	Route	Rôle
<code>index(Request)</code>	GET /api/etablisements	Liste paginée + filtres.
<code>show(int \$id)</code>	GET /api/etablisements/{id}	Détail + nb d'utilisateurs.
<code>update(Request, \$id)</code>	PUT /api/etablisements/{id}	Valide (nom, adresse, ville, pays, telephone, email) puis met à jour.
<code>toggleActif(int \$id)</code>	PATCH /api/etablisements/{id}/toggle-actif	Active/désactive.
<code>statistiques()</code>	GET /api/etablisements/statistiques	KPIs plateforme.

1.6 Les modèles du socle

Rappel : un **modèle** Eloquent = une classe PHP qui représente une table. Ses propriétés clés : `$table` (nom de table), `$fillable` (colonnes remplitables en masse), `$hidden` (colonnes cachées dans le JSON), `$casts` (conversion auto de types), et des **relations** (méthodes `belongsTo`, `hasMany`, `belongsToMany`).

app/Models/Utilisateur.php (le compte de connexion)

Étend `Authenticatable` et implémente **JWTSubject** (contrat exigé par `jwt-auth`). - `getAuthPassword(): string` -> indique que le mot de passe est dans `mot_de_passe_hash` (et non la colonne `password` par défaut). - `getJWTIdentifier(): mixed` -> l'id mis dans le jeton (la clé primaire). - `getJWTCustomClaims(): array` -> **claims** ajoutés au jeton : `uuid`, `email`, `role (code)`, `etablissement_id`. C'est ainsi que le tenant voyage dans le jeton. - `$hidden` : `mot_de_passe_hash`, `token_verification`, `token_reset_mdp` (jamais exposés). - `$casts` : booléens (`est_actif`, `email_verifie`), dates, `tentatives_connexion` en entier. - `boot()` génère l'`uuid` à la création. - Relations : `etablissement()`, `role()`, `permissionsGranulaires()` (overrides), `sessions()`, `journalConnexions()`. - Helpers : `estBloque()` (compte temporairement bloqué ?), `estSuperAdmin()` (`role.code === 'super_admin'`), `nomComplet()` (« prénom nom »).

Récapitulatif des 7 modèles du socle

Modèle	Table	Points notables
<code>Utilisateur</code>	<code>utilisateurs</code>	compte + <code>JWTSubject</code> (voir ci-dessus)
<code>Etablissement</code>	<code>etablisements</code>	tenant ; <code>estValide()</code> ; relations utilisateurs/roles/demandes
<code>Role</code>	<code>roles</code>	<code>est_système</code> protégé ; <code>permissions()</code> ; <code>codesPermissions()</code>
<code>Permission</code>	<code>permissions</code>	code/module ; sans timestamps
<code>SessionUtilisateur</code>	<code>sessions</code>	sessions JWT actives ; <code>estExpiree()</code> ; sans timestamps

JournalConnexion	journal_connexions	audit des connexions ; sans timestamps
DemandeCompte	demandes_compte	donnees en JSON ; états estEnAttente/Validee/Rejetee

1.7 Les tables (migrations) du socle

Conforme au schéma `bd-Edusphere_v2_0` (§1-9), avec horodatage Laravel standard.

- **etablisements** : id, uuid, nom, adresse, ville, pays (déf. 'Cameroun'), telephone, email, est_actif.
- **permissions** : id, code (unique), libelle, module, description.
- **roles** : id, etablisement_id (null = rôle système global), nom, code, description, est_système. Unicité (etablisement_id, code).
- **role_permissions** : pivot (role_id, permission_id).
- **utilisateurs** : id, uuid, etablisement_id (null pour super admin), role_id, nom, prenom, email (unique), mot_de_passe_hash, telephone, photo_url, genre, date_naissance, est_actif, email_verifie, tokens divers, derniere_connexion, tentatives_connexion, bloque_jusqu_a.
- **utilisateur_permissions** : override (utilisateur_id, permission_id, type_acces dans {accorder,retirer}, accorde_par, accorde_le).
- **sessions** : sessions JWT (utilisateur_id, token, ip_adresse, agent_navigateur, appareil, est_active, expire_le, ferme_le).
- **journal_connexions** : audit (utilisateur_id, session_id, type_evenement, ip_adresse, agent_navigateur, detail).
- **journal_audit** : audit générique des actions (table prévue au schéma).
- **demandes_compte** (*ajout de Benjo, hors schéma initial*) : type_demande, donnees (JSON), etablisement_id, statut, traite_par, traite_le, motif_rejet, utilisateur_cree_id, etablisement_cree_id, soumis_le.

1.8 Les routes du socle (`routes/api.php`)

Toutes sous `/api` (le préfixe `v1` a été retiré). Les routes protégées passent par `['auth:api', 'etablisement.scope']`, puis éventuellement `permission:<code>`.

PUBLIC		
POST	<code>/api/auth/login</code>	<code>LoginController@login</code>
POST	<code>/api/demandes/etablisement</code>	<code>DemandeController@soumettreEtablissement</code>
PROTÉGÉ (auth:api + etablisement.scope)		
POST	<code>/api/auth/logout</code>	<code>LoginController@logout</code>
POST	<code>/api/auth/refresh</code>	<code>LoginController@refresh</code>
GET	<code>/api/auth/me</code>	<code>LoginController@moi</code>
Demandes (permission:utilisateurs.creer sauf mention) :		
GET	<code>/api/demandes</code>	<code>DemandeController@index</code>
GET	<code>/api/demandes/statistiques</code>	<code>DemandeController@statistiques</code>
GET	<code>/api/demandes/{id}</code>	<code>DemandeController@show</code>
POST	<code>/api/demandes/enseignant</code>	<code>@soumettreEnseignant</code> (permission:enseignants.creer)
POST	<code>/api/demandes/etudiant</code>	<code>@soumettreEtudiant</code> (permission:etudiants.creer)
POST	<code>/api/demandes/{id}/valider</code>	<code>DemandeController@valider</code>
POST	<code>/api/demandes/{id}/rejeter</code>	<code>DemandeController@rejeter</code>
Établissements :		
GET	<code>/api/etablisements</code>	(permission:etablisements.voir)
GET	<code>/api/etablisements/statistiques</code>	(permission:etablisements.voir)
GET	<code>/api/etablisements/{id}</code>	(permission:etablisements.voir)

```

PUT    /api/etablisements/{id}                (permission:etablisements.modifier)
PATCH /api/etablisements/{id}/toggle-actif (permission:etablisements.modifier)

Rôles & permissions :
GET    /api/permissions                (permission:utilisateurs.permissions)
GET    /api/roles                      (permission:utilisateurs.voir)
GET    /api/roles/{id}                 (permission:utilisateurs.voir)
POST   /api/roles                      (permission:utilisateurs.permissions)
PUT    /api/roles/{id}                 (permission:utilisateurs.permissions)
DELETE /api/roles/{id}                 (permission:utilisateurs.permissions)
GET    /api/utilisateurs/{id}/permissions
POST   /api/utilisateurs/{id}/permissions
DELETE /api/utilisateurs/{id}/permissions/{code}

```

Providers

`bootstrap/providers.php` ne charge que `App\Providers\AppServiceProvider`. Les services (`AuthService`, `PermissionService...`) n'ont **pas besoin** d'être déclarés : Laravel les **résout automatiquement** via l'injection par constructeur (auto-wiring du conteneur).

Partie 2 - Module #1 « Structure Académique » (notre lot)

Hiérarchie académique : **Année académique** et **Faculté** -> **Département** -> **Filière** -> **Niveau** -> **Classe**, plus les **Semestres**. C'est le socle de plusieurs autres modules (cours, étudiants, examens...).

2.1 L'infrastructure multi-tenant automatique

Contrairement au socle (filtrage manuel), nos modèles filtrent **automatiquement** par établissement. Trois pièces :

`app/Support/ContexteTenant.php`

Petit registre statique de l'établissement courant, utile **hors HTTP** (seeders, commandes, tests) où il n'y a pas de jeton JWT. - `definir(?int $id): void` -> fixe l'établissement courant. - `id(): ?int` -> le lit. - `reinitialiser(): void` -> le remet à null.

`app/Models/Scopes/PorteeEtablissement.php` (scope global)

Implémente l'interface `Scope` de Laravel. Sa méthode `apply(Builder, Model)` : 1. Résout l'établissement : `ContexteTenant::id()` **sinon** `auth('api')->user()->etablissement_id`. 2. Si trouvé, ajoute `WHERE etablissement_id = <id>` à **toute** requête du modèle. 3. Si rien (ex. super admin, console sans contexte), aucun filtre.

`app/Models/Concerns/AppartientEtablissement.php` (trait)

À ajouter (use) sur tout modèle ayant une colonne `etablissement_id`. Sa méthode `bootAppartientEtablissement()` (appelée auto par Eloquent) : 1. Enregistre le scope global `PorteeEtablissement` (lecture filtrée). 2. Sur l'événement `creating`, **injecte** `etablissement_id` depuis le contexte si absent - le code métier n'a donc jamais à le renseigner.

■ Bénéfice : isolation **par défaut**. Impossible d'oublier le filtre => pas de fuite entre tenants.

2.2 Les 7 modèles (tous avec `AppartientEtablissement`)

Modèle	Table	\$fillable (hors <code>etablissement_id</code>)	Relations
AnneeAcademique	annees_academiques	libelle, date_debut, date_fin, est_active	semestres(), classes()
Faculte	facultes	nom, code, est_actif	departements()
Departement	departements	faculte_id, nom, code, chef_id, est_actif	faculte(), chef(), filieres()
Filiere	filieres	departement_id, nom, code, type_formation, duree_annees, est_actif	departement(), niveaux()
Niveau	niveaux	filiere_id, libelle, ordre	filiere(), classes()
Classe	classes	niveau_id, annee_academique_id, nom, capacite_max, est_actif	niveau(), anneeAcademique()
Semestre	semestres	annee_academique_id, libelle, date_debut, date_fin, est_actif	anneeAcademique()

Conversions (\$casts) : booléens (`est_actif/est_active`), dates (`date_debut/date_fin`), entiers (`ordre`, `duree_annees`, `capacite_max`).

2.3 Les 7 tables (migrations)

Toutes portent `etablissement_id` (FK -> `etablissements`, suppression en cascade) - choix de **dénormalisation** pour un filtrage tenant uniforme et performant (conforme à `ARCHITECTURE.md`). Chaîne des dépendances : `annees_academiques` -> `facultes` -> `departements` -> `filieres` -> `niveaux` -> `semestres` -> `classes`. Particularités : `annees_academiques` unique sur (`etablissement_id`, `libelle`) ; `departements.chef_id` -> `utilisateurs` (nullable) ; `classes` relie un niveau à une `annee_academique`.

2.4 Seeder

`database/seeder/StructureAcademiqueSeeder.php` : crée (idempotent, via `firstOrCreate`) un établissement démo, une année active + 2 semestres, une faculté -> département -> filière, et les niveaux L1-L3 avec leurs classes. Il pose `ContexteTenant::definir($etab->id)` pour que `etablissement_id` soit injecté automatiquement, puis `reinitialiser()`. Lancement : `php artisan db:seed --class=StructureAcademiqueSeeder`.

2.5 Couche API (CRUD)

Chaque entité a un **contrôleur** (`app/Http/Controllers/Structure/`) et un **FormRequest** (`app/Http/Requests/Structure/`). Les routes sont déclarées en `apiResource` (`param {id}`), sous `['auth:api', 'etablissement.scope']`.

Patron commun des 5 méthodes (toutes renvoient `{succes, donnees|message}`) : - `index` : liste **paginée** (`par_page`, déf. 20), recherche `recherche` (`ilike`) et filtres par parent. - `store` : crée (201). `etablissement_id` est injecté automatiquement par le trait. - `show` : détail + relations (`findOrFail`, **scopé tenant** -> 404 si autre établissement). - `update` / `destroy` : modifie / supprime (toujours **scopé tenant**).

L'isolation est **automatique** : le scope global `PorteeEtablissement` filtre toutes les requêtes, y compris `findOrFail`. Inutile de filtrer à la main dans les contrôleurs.

Validation (FormRequests) - tous étendent `StructureRequest` (qui fournit `authorize()` et `etablissementCourant()`). Les clés étrangères parentes sont validées avec une règle `exists` **bornée au tenant** (ex. un `faculte_id` doit appartenir à l'établissement courant) ; `annees_academiques.libelle` est unique par établissement.

Routes exposées (toutes en `/api`, protégées) : | Ressource | Endpoints | ---|---| | `annees-academiques` | `GET/POST /api/annees-academiques`, `GET/PUT/PATCH/DELETE /api/annees-academiques/{id}` | | `facultes` | idem sur `/api/facultes` | | `departements` | idem sur `/api/departements` (filtre `faculte_id`) | | `filieres` | idem sur `/api/filieres` (filtre `departement_id`) | | `niveaux` | idem sur `/api/niveaux` (filtre `filiere_id`) | | `classes` | idem sur `/api/classes` (filtres `niveau_id`, `annee_academique_id`) | | `semestres` | idem sur `/api/semestres` (filtre `annee_academique_id`) |

RBAC : ces routes seront en plus protégées par `permission:<code>` (ex. `structure.gerer`) une fois les codes de permission semés côté socle (coordination avec Benjo).

Annexes

A. Format des réponses JSON

- Succès : { "succes": true, "message": "...", "donnees": ... }
- Erreur : { "succes": false, "message": "...", "code": "..." }
- Les listes sont **paginées** (objet Laravel `LengthAwarePaginator` : `data`, `current_page`, `total...`).

B. Codes d'erreur applicatifs

Code	Sens
UNAUTHENTICATED	jeton absent/invalid/expiré
ACCOUNT_DISABLED	compte est_actif=false
SUBSCRIPTION_INACTIVE	établissement désactivé
FORBIDDEN	permission manquante

C. Glossaire

- **Tenant** : un établissement. Multi-tenant = une base partagée, données isolées par `etablissement_id`.
- **JWT** : jeton signé d'authentification, sans état côté serveur.
- **RBAC** : contrôle d'accès par rôles + permissions.
- **Middleware** : filtre exécuté avant le contrôleur.
- **Service** : classe qui porte la logique métier.
- **FormRequest** : classe qui valide les données entrantes.
- **Modèle / Eloquent** : classe table ; ORM de Laravel.
- **Scope global** : filtre SQL appliqué automatiquement à un modèle.
- **Trait** : bloc de code réutilisable « collé » dans une classe (`use`).
- **Migration** : script versionné qui crée/modifie une table.
- **Seeder / Factory** : remplissage de données (démon) / générateur de fausses données.
- **Job / Queue** : tâche exécutée en arrière-plan (file d'attente).

- **Event** : signal métier émis pour déclencher des réactions découplées.

Document généré pour l'apprentissage du backend EduSphere. Mis à jour au fil des modules.